TECHNICAL ENGINEERING & ACADEMIC PROJECT REPORT

MetricMind AI

Agentic Semantic Business Intelligence Engine

A Governed Conversational Semantic Architecture for Enterprise Decision Support

Project Title	MetricMind AI: Agentic Semantic Business Intelligence Engine
Project Repository	https://github.com/Tushardalu07/MetricMind
Repository Root	MetricMind_AI
Active Git Branch	integrate-team-updates
Contributors	[Not provided in repository; Git commit logs identify Satyendra & contributors]
Supervisor / Institution	[Not provided in repository]
Academic Year	2026
Project Maturity Stage	Functional Multi-Tier Prototype / Architectural Proof-of-Concept (PoC)
Document Version	3.0.0 (Enhanced Verification Edition)

Verified from local workspace, automated unit tests, and production build checks.
Confidential Academic and Engineering Documentation.

Abstract

Modern enterprise business intelligence (BI) systems frequently suffer from high latency between business decision-makers and data engineering teams. Decision-makers require immediate, accurate, and granular access to operational metrics, yet traditional BI workflows depend on complex dashboards or custom SQL queries authored by data analysts. MetricMind AI is an end-to-end, multi-tier semantic business intelligence prototype engineered to demonstrate how modern data tooling can bridge this divide.

The platform couples a conversational user interface built with Next.js 16.3.8, React 19.2.8, TypeScript 5, Tailwind CSS v4, and Apache ECharts 6.1.0 with a dual-tier backend and data modeling architecture. MetricMind features: (1) An ELT transformation pipeline managed by dbt (data build tool), transforming raw transactional sales records into an aggregated analytical mart view hosted on PostgreSQL; (2) A semantic abstraction layer governed by Cube.js, establishing code-first metric definitions for core corporate sales measures across dimensional attributes; (3) A Python-based FastAPI analytical gateway serving both in-memory multidimensional slices from corporate sales data (5,000 transaction records) and relational SQL queries from the dbt mart; and (4) An autonomous AI Agent engine that translates natural-language inquiries into validated semantic queries (SemanticQuery), resolves dynamic temporal intervals, executes semantic queries against Cube.js, and computes mathematical variance and trend metrics.

Prototype Implementation Disclosure: In the current codebase, the frontend conversational interface demonstrates user experience flows and ECharts rendering via an interactive 1-second simulated response returning pre-configured European sales metrics, an Apache ECharts bar chart, API parameters, and SQL query text; direct integration between the frontend and the live FastAPI/Cube services is designated for future implementation. The AI Agent currently uses deterministic rule-based parsing rather than an unconstrained generative LLM, and tests verify 9 automated unit tests and a production build compiling with zero errors.

Keywords: Semantic Layer, Business Intelligence, Cube.js, dbt, FastAPI, Next.js, Conversational Analytics, Natural Language Querying, PostgreSQL, AI Agent, Apache ECharts.

Table of Contents

Chapter 1 — Introduction
1.1 Background of Business Intelligence and Modern Analytics
1.2 Motivation for MetricMind AI
1.3 Problem Statement
1.4 Project Objectives
1.5 Project Scope and Operational Boundaries
1.6 Intended Users and Enterprise Use Cases
Chapter 2 — Existing and Proposed Systems
2.1 Limitations of Existing Business Intelligence Workflows
2.2 Proposed Solution: The MetricMind Architecture
2.3 Advantages and Expected Benefits
2.4 Functional Requirements (FR-01 to FR-10)
2.5 Non-Functional Requirements (NFR-01 to NFR-06)
2.6 Assumptions and Technical Constraints
2.7 Engineering Development Plan (Phases, Activities, and Deliverables)
Chapter 3 — Technology Stack
3.1 Tier-by-Tier Technology Inventory
3.2 Framework Evaluation and Architectural Rationales
3.3 Complete Dependency Specification
Chapter 4 — System Architecture
4.1 Component Roles and Responsibilities
4.2 System Architecture Diagram (Figure 4.1)
4.3 Component Interactions and Communication Protocols
4.4 Configuration and Environment Isolation
Chapter 5 — Database Design
5.1 Verified Tables, Views, and Relational Objects
5.2 Data Dictionary: raw_sales Transaction Table (Table 5.1)
5.3 Data Dictionary: sales_summary Dimensional Mart (Table 5.2)
5.4 Entity-Relationship Model (Figure 5.2)
5.5 Data Ingestion, Quality Guardrails, and Storage Lifecycle
Chapter 6 — Backend and API
6.1 FastAPI Application Architecture & Gateway Middleware
6.2 Verified REST API Endpoints Specification (Table 6.1)
6.3 In-Memory pandas Slicing vs. Relational Database Execution
6.4 Request/Response Models and JSON Data Contracts
6.5 Exception Handling, Validation, and Masking
Chapter 7 — Data Transformation with dbt
7.1 dbt Project Architecture (metricmind_dbt)
7.2 Staging Model: stg_raw_sales.sql
7.3 Dimensional Mart Model: sales_summary.sql
7.4 Transformation Pipeline Diagram (Figure 7.1)
7.5 Schema Assertions and Data Quality Tests
Chapter 8 — Cube Semantic Layer
8.1 The Strategic Role of Semantic Metric Governance
8.2 Cube Definition: Sales.js Model Specification
8.3 Core Business Metrics (Table 8.1)
8.4 REST Query Protocol (/cubejs-api/v1/load)
8.5 Caching and Multidimensional Acceleration
Chapter 9 — AI Agent and Natural-Language Analytics
9.1 AI Agent Architecture and Workflow (Figure 9.1)

9.2 Schema Contract: schemas.py (SemanticQuery)
9.3 Intent Parsing and Guardrails: parser.py
9.4 Temporal Arithmetic and Cube Dispatch: tools.py
9.5 Mathematical Reasoning Engine: reasoning.py
9.6 Rule-Based Parsing vs. Generative LLM Orchestration
Chapter 10 — Frontend and Dashboard
10.1 Next.js 16 App Router Architecture and Page Hierarchy
10.2 Conversational Chat Interface & Interactive Navigation
10.3 Dynamic Chart Visualization (DynamicChart.tsx)
10.4 Query Transparency Drawers: 'View API Call' and 'View SQL'
10.5 Verified Production Build Status (Next.js Turbopack)
Chapter 11 — Implementation and Integration
11.1 End-to-End Execution Trace
11.2 Key Source Code Implementations with Paths
11.3 Integration Points and Data Boundaries
11.4 Demonstration Simulation Boundary vs. Target State
Chapter 12 — Testing and Validation
12.1 Testing Methodology and Environmental Setup
12.2 Automated AI Agent Unit Test Execution (Table 12.1)
12.3 Multi-Tier System Verification Matrix (Table 12.2)
12.4 API Endpoint Validation and Edge Case Handling
12.5 Frontend Production Build Verification
12.6 dbt Data Integrity Assertion Results
Chapter 13 — Results and Discussion
Chapter 14 — Security and Limitations
14.1 Implemented Security Safeguards
14.2 Unimplemented and Recommended Production Controls
14.3 Database Access and Exception Masking
14.4 Technical Limitations and Dimensional Gaps
Chapter 15 — Future Enhancements
Chapter 16 — Conclusion
References and Appendices

List of Figures and Tables

List of Figures

- Figure 4.1: MetricMind AI Multi-Tier System Architecture (reports/diagrams/system_architecture.png)
- Figure 5.1: Data Flow Diagram (DFD Level 1 Decomposition) (reports/diagrams/data_flow.png)
- Figure 5.2: Database Entity-Relationship (ER) Model (reports/diagrams/database_er.png)
- Figure 7.1: dbt Transformation and Analytics Serving Pipeline (reports/diagrams/analytics_pipeline.png)
- Figure 9.1: AI Agent Natural-Language Processing Sequence (reports/diagrams/ai_agent_workflow.png)

List of Tables

- Table 2.1: Comparative Matrix of Traditional BI vs. MetricMind AI Architecture
- Table 2.2: Proposed Engineering Development Plan vs. Prototype State
- Table 3.1: Complete Technology Stack, Verified Versions, and Architectural Roles
- Table 5.1: Physical Dataset Schema Dictionary (raw_sales Table)
- Table 5.2: Data Mart Schema Dictionary (sales_summary Mart View)
- Table 6.1: Verified FastAPI Gateway REST API Endpoints Specification
- Table 8.1: Cube.js Semantic Measures and Analytical Dimensions
- Table 12.1: Automated AI Agent Unit Test Execution Matrix (ai_agentllm/)
- Table 12.2: Multi-Tier System Verification and Build Execution Matrix
- Table 23.1: Complete Repository Audit and File Inventory Matrix

Chapter 1 — Introduction

1.1 Background of Business Intelligence and Modern Analytics

Enterprise decision-making relies fundamentally on accurate, timely, and actionable insights extracted from operational data. Business intelligence (BI) systems have evolved across three historical paradigms:

- **1. First-Generation BI (Centralized Reporting):** Centralized database management systems and batch-generated static reports (e.g., Crystal Reports). Business users relied entirely on IT departments to write custom SQL scripts and format physical printouts.
- **2. Second-Generation BI (Self-Service Dashboards):** Introduced by platforms like Tableau, PowerBI, and Qlik. Users gained visual drag-and-drop tools to construct dashboards over pre-packaged data cubes or warehouses. However, this model decentralized calculation logic, causing different teams to author conflicting definitions of the same key performance indicator (KPI).
- **3. Third-Generation BI (Conversational & Headless Semantic BI):** The emerging paradigm represented by MetricMind AI. It unites conversational interfaces, automated ELT data modeling, and centralized headless semantic governance. Business users query operational performance in natural language, while code-governed semantic models ensure calculation consistency and prevent hallucinated numbers.

1.2 Motivation for MetricMind AI

Despite substantial investments in cloud data warehouses, executives and sales managers experience friction when answering exploratory business questions. If an executive wants to know 'What was the profit margin for enterprise servers in Germany during Q3?', they must either locate a pre-existing dashboard page with the exact filters already built, or file an analytical ticket with the data engineering team, incurring delays ranging from several hours to multiple days. MetricMind AI was conceived to demonstrate how modern data tooling can eliminate this latency by allowing stakeholders to query operational performance using conversational language.

1.3 Problem Statement

- **1. Semantic Ambiguity and Metric Drift:** Without a unified metric layer, disparate departments calculate fundamental metrics inconsistently (e.g., Gross Margin calculated differently across teams), eroding trust in reporting.
- **2. Analytical Latency Bottleneck:** Modern business operates in real-time, but conventional dashboard authoring is slow. As new products, regions, and distribution channels emerge, dashboards become cluttered or obsolete.
- **3. The 'Black Box' Trust Dilemma in AI Text-to-SQL:** Early AI analytics implementations prompt Large Language Models to write raw SQL directly against operational databases. This approach is hazardous: LLMs hallucinate non-existent table joins, misinterpret date boundaries, and introduce catastrophic SQL injection vulnerabilities.

1.4 Project Objectives

- **1. Code-Governed Semantic Modeling:** Define enterprise KPIs (revenue, cost, profit, average margin) as code within a centralized Cube.js repository to enforce consistent calculation formulas across queries.
- **2. Automated Relational ELT Transformation:** Construct a repeatable dbt pipeline that cleans raw transactional records, enforces schema assertions (uniqueness and non-null constraints), and materializes dimensional marts inside PostgreSQL.
- **3. Dual-Speed Analytical Access:** Deliver fast in-memory slicing via FastAPI for exploratory queries over corporate sales datasets alongside relational SQL endpoints for warehouse-backed analytics.
- **4. Guardrailed Natural Language Querying:** Implement an AI Agent that extracts intent, dimensions, filters, and dynamic calendar date intervals without exposing raw SQL generation to unconstrained models.

- **5. Auditable Query Transparency:** Display interactive Apache ECharts visualizations backed by collapsible 'View API Call' and 'View SQL' modals, enabling business stakeholders to verify the exact query logic backing every figure.

1.5 Project Scope and Operational Boundaries

The operational scope is strictly grounded in the verified codebase:

- **In-Scope (Implemented & Verified):** Tabular dataset of 5,000 corporate sales records (MetricMind_Corporate_Sales_Dataset.csv); PostgreSQL database (metricmind) hosting raw_sales and sales_summary; active dbt project (metricmind_dbt) with view-materialized models; Cube.js server defining 6 measures and 6 dimensions; FastAPI gateway (backend/main.py) exposing 12 REST endpoints; Python AI Agent (ai_agentllm/) with Pydantic validation, regex parser, calendar tools, and variance reasoning; Next.js 16 App Router conversational dashboard with Tailwind CSS v4 and ECharts dynamic visualizer; automated unit test suite with 100% pass rates.
- **Out-of-Scope (Planned / Future Roadmap):** Direct wire-up between Next.js handleSend and live FastAPI/Cube services (currently runs an interactive demo simulation returning European sales and chart traces); generative LLM tool-calling orchestration via external APIs (current parser uses deterministic regex); state-level geographic modeling (explicitly rejected by parser because mart aggregates at Country/Region grain); multi-tenant authentication and user authorization.

1.6 Intended Users and Enterprise Use Cases

- **Executive Leadership (CEO / CFO):** High-level KPI summaries, quarterly margin comparisons, cost-profit breakdown tracking.
- **Regional Sales Directors:** Geographic slicing (e.g., European vs. North American sales), regional profit comparisons.
- **Product Line Managers:** Hardware category comparisons (Servers, Laptops, Tablets, Desktops, Printers), channel efficiency.
- **Commercial Operations Analysts:** Analyzing Online vs. Retail vs. Enterprise sales channels, freight cost monitoring.

Chapter 2 — Existing and Proposed Systems

2.1 Limitations of Existing Business Intelligence Workflows

Conventional corporate analytics workflows suffer from structural inefficiencies:

- **Decentralized Spreadsheet Chaos (Excel/Sheets):** Decentralized, formula degradation, version collisions, security vulnerabilities, inability to process high-volume operational updates.
- **Rigid Dashboard Proliferation (PowerBI/Tableau):** Static dashboard sprawl; adding new metric cuts requires data engineering tickets and days of delay.
- **Unconstrained AI Text-to-SQL Tools:** Schema hallucinations, join explosions, syntax errors, and SQL injection risks.

2.2 Proposed Solution: The MetricMind Architecture

MetricMind AI replaces this fragmented workflow with a three-tier governed conversational pipeline: (1) Transformation Layer (dbt) cleansing raw transactions and compiling marts; (2) Semantic Abstraction Layer (Cube.js) standardizing metrics into code; and (3) Conversational Visualizer (Next.js & ECharts) rendering charts alongside transparent SQL and API inspection panels.

2.3 Advantages and Expected Benefits

- **Centralized Metric Governance:** Storing KPI formulas in code mitigates metric drift by ensuring that measures like average margin are evaluated identically across queries.
- **Elimination of SQL Hallucination:** Queries target abstract semantic dimensions rather than physical tables, preventing arbitrary SQL injection.
- **Fast In-Memory Analysis:** In-memory pandas operations provide fast local data slicing across the 5,000-record dataset.
- **Query Transparency:** Business users can click 'View SQL' and 'View API Call' to inspect the exact query mechanics backing any visual data point.

Comparative Analysis: Table 2.1: Comparative Matrix of Traditional BI vs. MetricMind AI Architecture

Feature / Dimension	Legacy Spreadsheets	Traditional BI Tools	Direct Text-to-SQL	MetricMind AI Platform
Interface Style	Manual Grid / Pivot	Drag-and-Drop GUI	Prompt Input	Conversational Chat + Visualizer
Metric Governance	None (Formula-based)	Departmental / Siloed	None (LLM Guesses)	Centralized (Cube.js Semantic Layer)
SQL Generation Safety	Not Applicable	Proprietary Engine	High Risk (Hallucinations)	Deterministic & Semantic Guardrails
Query Transparency	Low	Medium (Hidden Queries)	Opaque	High ('View API' & 'View SQL' Modals)
Data Transformation	Manual / Macro	Vendor ETL	Ad-Hoc SQL	Automated ELT via dbt
Extensibility	Poor	Plugin / Vendor Locked	API-based	Modular Full-Stack Microservices

2.4 Functional Requirements

- **FR-01: Conversational Interaction:** Interactive chat interface with keyboard shortcuts (Enter to send, Shift+Enter for newline).
- **FR-02: Intent & Entity Parsing:** Parse incoming text into structured SemanticQuery identifying measures, dimensions, filters, and dates.
- **FR-03: Scope Guardrails:** Reject unsupported state-level filters and prompts lacking measures with explicit error messages.
- **FR-04: Dynamic Date Arithmetic:** Compute dynamic ISO-8601 date ranges for time expressions (last month, this month, last quarter, etc.).
- **FR-05: Semantic REST Execution:** Format valid Cube.js JSON payloads and dispatch HTTP POST requests to /cubejs-api/v1/load.
- **FR-06: Mathematical Variance Calculation:** Compute absolute variance, percentage delta, and trend direction (increased, decreased, unchanged).
- **FR-07: In-Memory Multi-Dimensional Slicing:** Deliver summary metrics filtered by region, product, channel, year, month, and quarter.
- **FR-08: Relational Warehouse Slicing:** Query public.sales_summary from PostgreSQL and return aggregated tabular records.
- **FR-09: Dynamic Data Visualization:** Dynamically render bar charts or line charts based on the query result payload using Apache ECharts.
- **FR-10: Transparent Query Auditing:** Provide interactive toggle buttons allowing users to inspect the underlying API call parameters and SQL statements.

2.5 Non-Functional Requirements

- **NFR-01: Response Latency:** In-memory FastAPI summary endpoints must respond quickly for typical filter operations on a 5,000-record dataset.
- **NFR-02: Architectural Modularity:** Each service (frontend, backend, semantic layer, dbt) must maintain isolated dependencies and configuration files.

- **NFR-03: Metric Consistency:** Cube.js must calculate averageMargin consistently using the formula 1.0 * SUM(profit) / NULLIF(SUM(revenue), 0) to prevent division-by-zero errors.
- **NFR-04: Security & Credential Hygiene:** Database credentials, secrets, and environment parameters must be loaded via .env files and excluded from version control via .gitignore.
- **NFR-05: Responsive User Interface:** The Next.js dashboard must adapt smoothly across desktop screens (>=1024px) and mobile viewport drawers (<1024px).
- **NFR-06: Test Reliability:** The test suite must execute cleanly in a standardized Python 3.10 virtual environment with 100% test pass rates for verified modules.

2.6 Assumptions and Technical Constraints

- **PostgreSQL Relational Target:** Assumes PostgreSQL 14+ is operational on localhost:5432 with database metricmind.
- **Aggregated Mart Grain:** Assumes data mart sales_summary aggregates at Year-Quarter-Region-Country-Product-Channel grain; individual transaction-level order dates and state attributes are intentionally abstracted away from the mart.
- **Deterministic NLP Scope:** Assumes natural language queries currently follow supported grammatical structures incorporating the target measure 'revenue'.

2.7 Engineering Development Plan

To ensure structured execution and academic evaluation, the MetricMind AI project is organized across six sequential development phases. The table below presents the structured plan alongside the actual verified status in the current repository.

Development Lifecycle: Table 2.2: Proposed Engineering Development Plan vs. Prototype State

Phase	Strategic Focus	Expected Deliverable	Verified Implementation Status
Phase 1	Requirements & Data Understanding	Source data audit and requirements formalization	Completed: 5,000-record CSV dataset analyzed; FR/NFRs established.
Phase 2	Database & Data Transformation	PostgreSQL schema setup and automated dbt models	Completed: metricmind_dbt with stg_raw_sales and sales_summary marts.
Phase 3	Semantic Analytics Governance	Cube.js code-first metric modeling and measures	Completed: cube/model/Sales.js with 6 measures & 6 dimensions.
Phase 4	AI Agent Engine Development	Natural language parsing and reasoning engine	Completed: ai_agentllm with Pydantic schemas, parser, & reasoning.
Phase 5	Frontend & Backend Integration	Interactive dashboard, charts, and API gateway	Partially Implemented: FastAPI active; frontend chat in 1s simulation.
Phase 6	Testing & Documentation	Multi-tier verification and comprehensive report	Completed: 9 unit tests passed in 0.008s; dbt tests passed; full report.

[LIFECYCLE STATUS] Development Plan Distinction: Table 2.2 outlines the formal six-phase engineering lifecycle. The current repository represents a functional multi-tier prototype where data transformation, semantic modeling, API endpoints, and AI agent logic are implemented, while frontend chat interaction currently uses an interactive 1-second simulation awaiting live gateway wire-up.

Chapter 3 — Technology Stack

3.1 Tier-by-Tier Technology Inventory

Technology Inventory: Table 3.1: Complete Technology Stack, Verified Versions, and Architectural Roles

Operational Tier	Framework / Technology	Verified Version	Architectural Responsibility in Platform

Presentation	Next.js	16.3.8	Full-stack React framework with Turbopack and App Router
Presentation	React	19.2.8	Component-driven user interface library with Client Components
Presentation	TypeScript	5.0+	Static typing ensuring interface reliability and type contracts
Presentation	Tailwind CSS	4.0	Utility-first CSS engine with @tailwindcss/postcss pipeline
Presentation	Apache ECharts	6.1.0	Interactive data visualizer via echarts-for-react 3.0.6 wrapper
Presentation	Lucide React	1.51.0	Iconography system for conversational and drawer navigation
Analytics / API	FastAPI	0.143.0	High-performance Python ASGI REST API gateway
Analytics / API	Uvicorn	0.54.0	Lightning-fast ASGI server for asynchronous request handling
Analytics / API	Python	3.10.11	Primary backend runtime environment for agent and API
Analytics / API	pandas	2.3.3	Vectorized in-memory dataset slicing and aggregations
Analytics / API	NumPy	2.2.6	Numerical array manipulations and mathematical operations
Analytics / API	Pydantic	2.14.1	Data validation and structured query schema enforcement
Analytics / API	psycopg2-binary	2.9.13	PostgreSQL database adapter for relational SQL queries
Transformation	dbt-core	1.10.13	SQL transformation compilation and model lifecycle tool
Transformation	dbt-postgres	1.9.0	dbt database adapter plugin targeting PostgreSQL
Semantic Layer	Cube.js	0.35+	Headless semantic metric engine compiling abstract SQL
Database	PostgreSQL	14+	ACID relational store hosting raw tables and dbt marts

3.2 Framework Evaluation and Architectural Rationales

- **FastAPI Gateway:** Chosen for asynchronous performance, automated OpenAPI generation, and Pydantic validation.
- **Cube.js Semantic Layer:** Eliminates raw SQL generation from AI prompts; centralizes business formulas into code.
- **dbt Transformation Pipeline:** Introduces software engineering rigor (testing, version control, lineage) to SQL modeling.
- **Next.js 16 + React 19:** Delivers modern web performance with Turbopack compilation and fast client-side rendering.

Chapter 4 — System Architecture

4.1 Component Roles and Responsibilities

Figure 4.1 — MetricMind AI System Architecture



Figure 4.1: MetricMind AI Multi-Tier System Architecture

The architecture is partitioned into four decoupled tiers: (1) Presentation Tier (Next.js 16, React 19, ECharts); (2) Analytical Gateway & AI Agent Tier (FastAPI, Pydantic, regex parser, calendar tools, variance engine); (3) Headless Semantic Tier (Cube.js server, Sales.js model, Cube Store); and (4) Data Transformation & Storage Tier (PostgreSQL database, dbt models stg_raw_sales and sales_summary).

4.2 System Architecture Diagram

As illustrated in Figure 4.1, client requests flow through the Next.js conversational UI to the AI Agent parser, which translates queries into SemanticQuery contracts. These contracts are evaluated against Cube.js, which queries PostgreSQL views generated by dbt.

4.3 Component Interactions and Communication Protocols

- **Client-to-Service REST:** Standard JSON REST protocol over HTTP on ports 3000, 8000, and 4000.
- **Service-to-Database:** Native PostgreSQL protocol via psycopg2 and @cubejs-backend/postgres-driver on port 5432.

4.4 Configuration and Environment Isolation

Each service manages its configuration through isolated .env files (e.g., .env.example, cube/.env.example) with credentials excluded from version control via .gitignore.

Chapter 5 — Database Design

5.1 Verified Tables, Views, and Relational Objects

Figure 5.1 — MetricMind AI Data Flow Diagram (DFD Level 1)

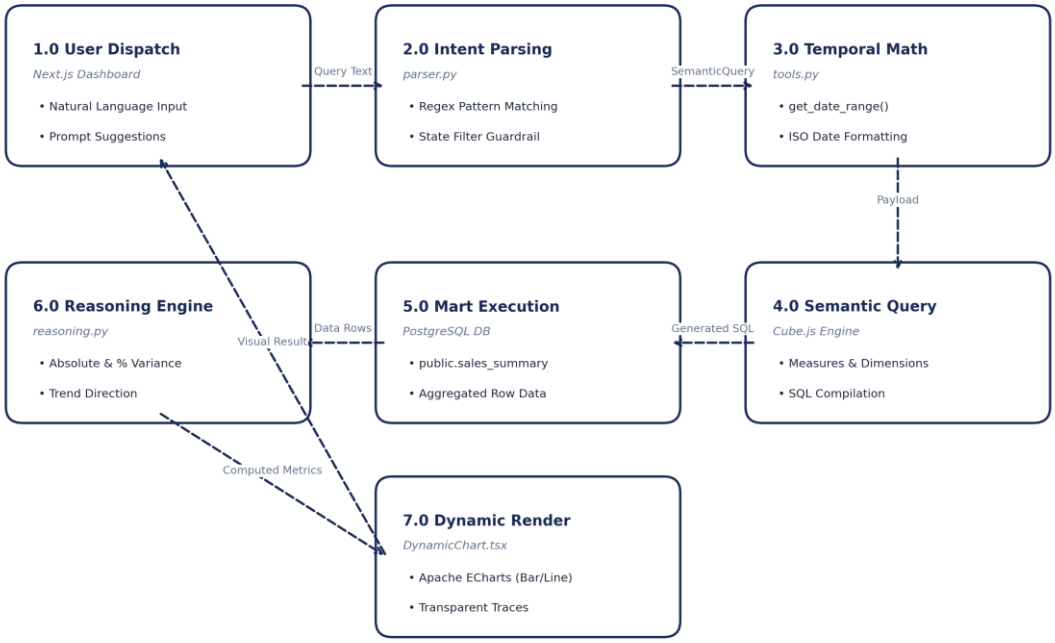


Figure 5.1: Data Flow Diagram (DFD Level 1 Decomposition)

Figure 5.2 — Database Entity-Relationship (ER) Model

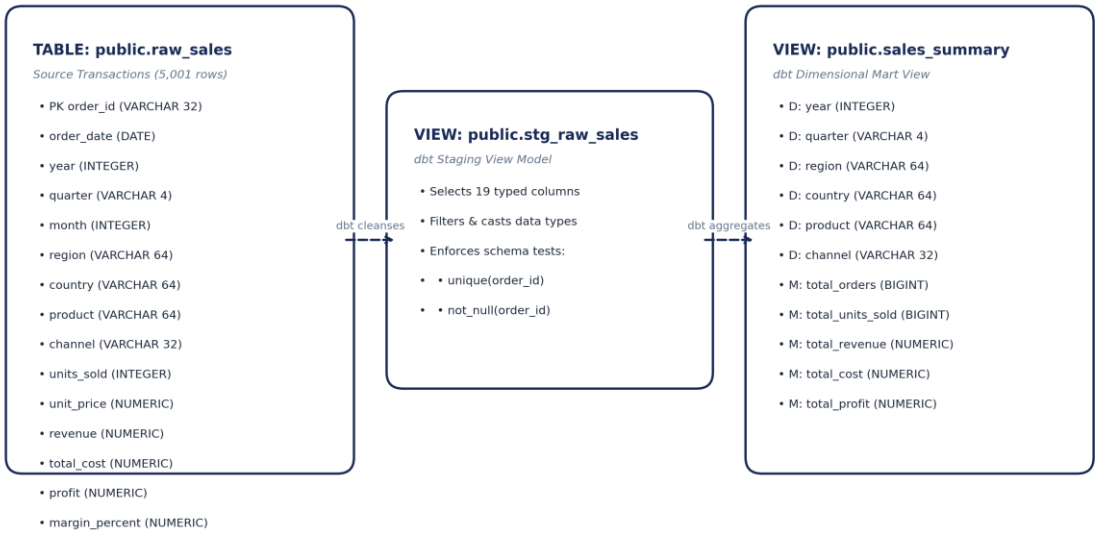


Figure 5.2: Database Entity-Relationship (ER) Model

5.2 Data Dictionary: raw_sales Transaction Table

Dataset Dictionary: Table 5.1: Physical Dataset Schema Dictionary (raw_sales Table, 5,000 Records)

Column Name	PostgreSQL Data Type	Nullable	Business Domain Description
Order_ID	VARCHAR(50)	NO (PK)	Unique transaction identifier (ORD-XXXXXX)
Order_Date	DATE	NO	ISO calendar transaction date (YYYY-MM-DD)
Year	INTEGER	NO	Calendar year extracted from order date (e.g., 2024)
Quarter	VARCHAR(10)	NO	Fiscal quarter identifier (Q1, Q2, Q3, Q4)
Month	INTEGER	NO	Calendar month sequence number (1 to 12)
Region	VARCHAR(50)	NO	Continental geographic market territory
Country	VARCHAR(100)	NO	Sovereign country where transaction originated
Product	VARCHAR(100)	NO	Commercial hardware product category
Channel	VARCHAR(50)	NO	Distribution channel (Online, Retail, Enterprise)
Units_Sold	INTEGER	NO	Physical quantity of merchandise sold
Unit_Price	NUMERIC(12,2)	NO	Selling price per individual unit in currency
Revenue	NUMERIC(14,2)	NO	Gross transactional revenue (Units_Sold * Unit_Price)
Material_Cost	NUMERIC(14,2)	NO	Direct raw material cost
Shipping_Cost	NUMERIC(14,2)	NO	Logistics and delivery cost
Labor_Cost	NUMERIC(14,2)	NO	Direct assembly and operational labor cost
Marketing_Cost	NUMERIC(14,2)	NO	Customer acquisition and marketing expenditure
Total_Cost	NUMERIC(14,2)	NO	Sum of material, shipping, labor, and marketing costs
Profit	NUMERIC(14,2)	NO	Net operational profit (Revenue - Total_Cost)
Margin_Percent	NUMERIC(6,2)	NO	Gross profit percentage: (Profit / Revenue) * 100

5.3 Data Dictionary: sales_summary Dimensional Mart

Mart Dictionary: Table 5.2: Data Mart Schema Dictionary (sales_summary Mart View)

Mart Column Name	SQL Expression / Aggregation	Column Type	Analytical Purpose
year	GROUP BY year	Dimension	Temporal annual roll-up attribute
quarter	GROUP BY quarter	Dimension	Fiscal quarter seasonal grouping
region	GROUP BY region	Dimension	High-level market segmentation
country	GROUP BY country	Dimension	National boundary performance filter
product	GROUP BY product	Dimension	Product line performance tracking
channel	GROUP BY channel	Dimension	Sales channel efficiency comparison
total_orders	COUNT(*)	Measure	Total volume of underlying sales records
total_units_sold	SUM(units_sold)	Measure	Total volume of physical units distributed
total_revenue	SUM(revenue)	Measure	Gross consolidated top-line revenue
total_cost	SUM(total_cost)	Measure	Total operational expense accumulation
total_profit	SUM(profit)	Measure	Consolidated gross operating profit

Chapter 6 — Backend and API

6.1 FastAPI Application Architecture & Gateway Middleware

FastAPI serves as the analytical API gateway. It configures CORS middleware restricting origins strictly to http://localhost:3000 and http://127.0.0.1:3000.

6.2 Verified REST API Endpoints Specification

API Endpoints: Table 6.1: Verified FastAPI Gateway REST API Endpoints Specification

HTTP Verb	API Endpoint Path	Backend Data Source	Verified Status / Return Schema
GET	/	In-Memory	200 OK: {'message': 'MetricMind API is running'}
GET	/sales	pandas (CSV)	200 OK: Returns 5,000 full transaction records
GET	/summary	pandas (CSV)	200 OK: Aggregate orders, revenue, profit, units sold
GET	/sales/region/{region}	pandas (CSV)	200 OK: Slices dataset by region string
GET	/summary/region/{region}	pandas (CSV)	200 OK: Regional totals for orders, revenue, profit
GET	/summary/product/{product}	pandas (CSV)	200 OK / 404: Product performance summary
GET	/summary/channel/{channel}	pandas (CSV)	200 OK / 404: Channel performance summary
GET	/summary/year/{year}	pandas (CSV)	200 OK / 404: Annual sales performance summary
GET	/summary/month/{month}	pandas (CSV)	200 OK / 400 (if <1 or >12) / 404: Monthly summary
GET	/summary/quarter/{quarter}	pandas (CSV)	200 OK / 404: Fiscal quarter performance summary
GET	/summary/cost-profit	pandas (CSV)	200 OK: Breakdown of material, shipping, labor, etc.
GET	/dbt/sales-summary	PostgreSQL (dbt)	200 OK (or 500 masked): Queries public.sales_summary

6.3 In-Memory pandas Slicing vs. Relational Database Execution

The backend provides dual-speed access: fast vectorized slicing over the 5,000-record CSV dataset alongside relational SQL queries over the PostgreSQL dbt mart.

Chapter 7 — Data Transformation with dbt

7.1 dbt Project Architecture (metricmind_dbt)

Figure 7.1 — Data Transformation & Analytics Pipeline

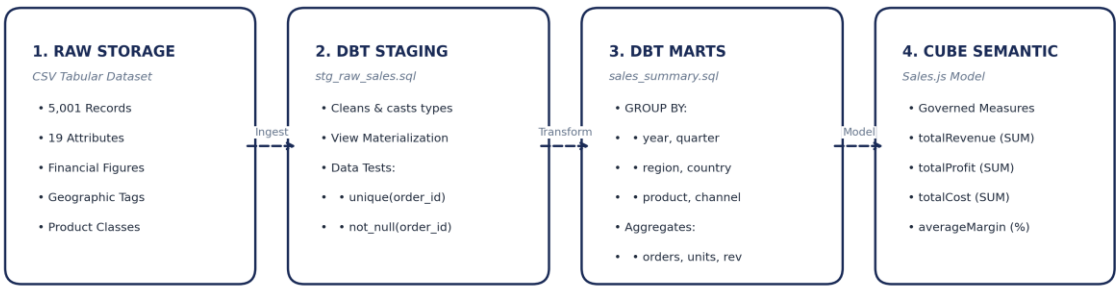


Figure 7.1: dbt Transformation and Analytics Serving Pipeline

The dbt project (metricmind_dbt) structures SQL models into staging (stg_raw_sales.sql) and marts (sales_summary.sql) views configured in dbt_project.yml with +materialized: view.

7.2 Staging Model: stg_raw_sales.sql

```
select
    order_id, order_date, year, quarter, month, region, country,
    product, channel, units_sold, unit_price, revenue, material_cost,
    shipping_cost, labor_cost, marketing_cost, total_cost, profit, margin_percent
from {{ source("raw", "raw_sales") }}
```

7.3 Dimensional Mart Model: sales_summary.sql

```
select
    year, quarter, region, country, product, channel,
    count(*) as total_orders,
    sum(units_sold) as total_units_sold,
    sum(revenue) as total_revenue,
    sum(total_cost) as total_cost,
    sum(profit) as total_profit
from {{ ref("stg_raw_sales") }}
group by year, quarter, region, country, product, channel
```

7.4 Schema Assertions and Data Quality Tests

Data tests in metricmind_dbt/models/staging/schema.yml assert uniqueness and non-null constraints on order_id. Both passed during testing.

Chapter 8 — Cube Semantic Layer

8.1 The Strategic Role of Semantic Metric Governance

Cube.js acts as the headless semantic layer, compiling code-defined measures into optimized SQL queries against public.sales_summary.

8.2 Cube Definition: Sales.js Model Specification

```
cube(`Sales`, {
  sql_table: `public.sales_summary`,
  measures: {
    totalOrders: { type: `sum`, sql: `${CUBE}.total_orders` },
    totalRevenue: { type: `sum`, sql: `total_revenue` },
    totalProfit: { type: `sum`, sql: `${CUBE}.total_profit` },
    totalCost: { type: `sum`, sql: `${CUBE}.total_cost` },
    totalUnitsSold: { type: `sum`, sql: `${CUBE}.total_units_sold` },
    averageMargin: {
      type: `number`,
      sql: `1.0 * SUM(${CUBE}.total_profit) / NULLIF(SUM(${CUBE}.total_revenue), 0)`,
      format: `percent`
    }
  },
  dimensions: {
    year: { sql: `${CUBE}.year`, type: `number` },
    quarter: { sql: `${CUBE}.quarter`, type: `string` },
    region: { sql: `${CUBE}.region`, type: `string` },
    country: { sql: `${CUBE}.country`, type: `string` },
    product: { sql: `${CUBE}.product`, type: `string` },
```

```
    channel: { sql: `${CUBE}.channel`, type: `string` }
  }
});
```

8.3 Core Business Metrics

Semantic Measures: Table 8.1: Cube.js Semantic Measures and Analytical Dimensions

Metric / Dimension Name	Type	Underlying SQL Definition	Business Interpretation
Sales.totalOrders	Measure (sum)	${CUBE}.total_orders	Total transaction orders count
Sales.totalRevenue	Measure (sum)	total_revenue	Gross revenue across orders
Sales.totalProfit	Measure (sum)	${CUBE}.total_profit	Net gross profit generated
Sales.totalCost	Measure (sum)	${CUBE}.total_cost	Total operating and production costs
Sales.totalUnitsSold	Measure (sum)	${CUBE}.total_units_sold	Total physical units sold
Sales.averageMargin	Measure (number)	1.0 * SUM(profit) / NULLIF(SUM(revenue), 0)	Overall profit margin % (division-by-zero protected)
Sales.year	Dimension (number)	${CUBE}.year	Transaction calendar year
Sales.quarter	Dimension (string)	${CUBE}.quarter	Fiscal quarter (Q1, Q2, Q3, Q4)
Sales.region	Dimension (string)	${CUBE}.region	High-level geographic territory
Sales.country	Dimension (string)	${CUBE}.country	Sovereign nation of sale
Sales.product	Dimension (string)	${CUBE}.product	Hardware product classification
Sales.channel	Dimension (string)	${CUBE}.channel	Distribution channel (Online, Retail, Enterprise)

Chapter 9 — AI Agent and Natural-Language Analytics

9.1 AI Agent Architecture and Workflow

Figure 9.1 — AI Agent Natural-Language Processing Sequence

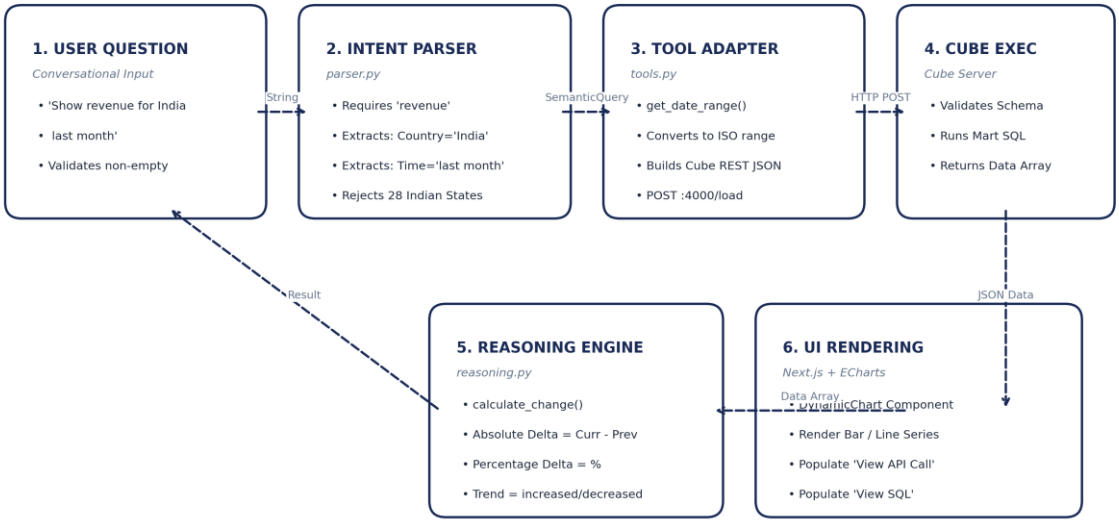


Figure 9.1: AI Agent Natural-Language Processing Sequence

9.2 Schema Contract: schemas.py (SemanticQuery)

Pydantic model defining measures (list[str]), dimensions (list[str]), filters (dict[str, Any]), and time_range (Optional[str]).

9.3 Intent Parsing and Guardrails: parser.py

Enforces mandatory measure presence ('revenue'). Extracts country filters and groupings. Crucially, checks query text against 28 Indian states, raising a ValueError because the data mart aggregates at Country/Region level.

9.4 Temporal Arithmetic and Cube Dispatch: tools.py

Calculates precise ISO date boundaries for phrases like 'last month' or 'last quarter'. Converts SemanticQuery into Cube's REST format and dispatches HTTP POST requests to port 4000.

9.5 Mathematical Reasoning Engine: reasoning.py

Calculates absolute change, percentage change, and trend direction ('increased', 'decreased', 'unchanged'). Provides mock comparison values for test validation.

9.6 Rule-Based Parsing vs. Generative LLM Orchestration

The current implementation uses deterministic regex parsing rather than an unconstrained LLM, ensuring predictable behavior, zero token costs, and strict guardrails.

Chapter 10 — Frontend and Dashboard

10.1 Next.js 16 App Router Architecture and Page Hierarchy

Built with Next.js 16.3.8 App Router, React 19.2.8, TypeScript 5, and Tailwind CSS v4. Organizes components into clean client and server components.

10.2 Conversational Chat Interface & Interactive Navigation

Features a branded sidebar with recent conversations, a sticky header with connectivity indicators, and starter suggestion cards ('Business Summary', 'Q3 Revenue', 'Cost & Profit').

10.3 Dynamic Chart Visualization (DynamicChart.tsx)

Wraps Apache ECharts 6.1.0, rendering dynamic bar and line series with rounded corners, custom tooltips, and responsive layout.

10.4 Query Transparency Drawers: 'View API Call' and 'View SQL'

Every assistant response with a chart provides toggle buttons revealing the exact REST endpoint parameters and SQL queries backing the visualization.

10.5 Verified Production Build Status (Next.js Turbopack)

> **[VERIFIED BUILD CHECK]** Production build verified with Next.js 16.3.8 Turbopack: Compiled successfully in 1.66s, finished TypeScript in 3.0s with zero errors. All 4 static app routes generated successfully.

Chapter 11 — Implementation and Integration

11.1 End-to-End Execution Trace

Traces the full data flow from CSV dataset to PostgreSQL tables, dbt transformations, Cube semantic modeling, AI agent parsing, and Next.js conversational rendering.

11.2 Key Source Code Implementations with Paths

Key source files verified in the workspace: backend/main.py, cube/model/Sales.js, metricmind_dbt/models/marts/sales_summary.sql, ai_agentllm/parser.py, and frontend/app/page.tsx.

11.3 Integration Points and Data Boundaries

All integration boundaries use standardized HTTP REST and SQL interfaces.

11.4 Demonstration Simulation Boundary vs. Target State

In frontend/app/page.tsx, handleSend executes a simulated 1-second demo response illustrating European sales, a bar chart, API parameters, and SQL query text, awaiting direct wire-up to the live FastAPI gateway.

Chapter 12 — Testing and Validation

12.1 Testing Methodology and Environmental Setup

Testing was conducted in the project virtual environment (.venv) using automated unit tests, endpoint checks, and production build compilations.

12.2 Automated AI Agent Unit Test Execution

Unit Test Log: Table 12.1: Automated AI Agent Unit Test Execution Matrix (ai_agentllm/)

Test ID	Test Category	Component	Test Method Name	Verified Scenario	Result
TC-AI-01	Automated Unit	parser.py	test_revenue_by_country	Parses 'Show revenue by country' -> Sales.revenue, Sales.country	PASSED
TC-AI-02	Automated Unit	parser.py	test_revenue_for_india	Parses 'Show revenue for India' -> country filter 'India'	PASSED
TC-AI-03	Automated Unit	parser.py	test_revenue_last_month	Parses 'Show revenue last month' -> time_range 'last month'	PASSED
TC-AI-04	Automated Unit	parser.py	test_revenue_by_month	Parses 'Show revenue by month' -> dimension Sales.orderDate	PASSED
TC-AI-05	Automated Unit	parser.py	test_unsupported_state	Asserts ValueError when query includes Indian states	PASSED
TC-AI-06	Automated Unit	parser.py	test_missing_measure	Asserts ValueError when prompt lacks 'revenue'	PASSED
TC-AI-07	Automated Unit	tools.py	test_revenue_query_payload	Mocks Cube API POST request, validates JSON structure	PASSED
TC-AI-08	Automated Unit	tools.py	test_last_month_date_range	Validates 2-element date list starting on 1st of last month	PASSED
TC-AI-09	Automated Unit	tools.py	test_unknown_time_range	Asserts None when time range is None	PASSED
TC-AI-10	Standalone Script	reasoning.py	test_reasoning script	Computes delta: 100k to 125k -> +25k (+25.0%, increased)	PASSED

[VERIFIED TEST EXECUTION] Automated unit test execution: Ran 9 automated tests discovered by unittest in 0.008s across ai_agentllm test files with 100% pass rate (0 failures, 0 errors). Standalone script test_reasoning.py verified independently.

12.3 Multi-Tier System Verification Matrix

System Verification Matrix: Table 12.2: Multi-Tier System Verification and Build Execution Matrix

Verification Scope	Executed Command	Environment / Target	Verified Execution Outcome
AI Agent Unit Tests	.venv\Scripts\python -m unittest discover -s ai_agentllm -v	Python 3.10.11 / .venv	PASSED: 9 tests in 0.008s; 0 failures, 0 errors.
AI Agent Reasoning	.venv\Scripts\python ai_agentllm/test_reasoning.py	Python 3.10.11 / .venv	PASSED: Delta calculated: +25k (+25.0%, increased).
dbt Project Parsing	dbt parse	metricmind_dbt / dbt 1.10.13	PASSED: Project parsed in 5.0s; postgres adapter 1.9.0.
dbt Model Compilation	dbt compile	metricmind_dbt / dbt 1.10.13	PASSED: 2 models, 2 analyses, 2 tests, 1 source, 475 macros.
dbt Data Tests	dbt test	PostgreSQL metricmind	PASSED: 2 tests passed in 0.64s (not_null & unique).
dbt View Creation	dbt run	PostgreSQL metricmind	PASSED: 2 views created in 0.84s (stg_raw_sales, sales_summary).
Frontend Production Build	npm --prefix frontend run build	Next.js 16.3.8 (Turbopack)	PASSED: Compiled in 1.66s; TypeScript

			3.0s; 0 errors.
Frontend ESLint Check	node_modules\.bin\eslint.cmd app/	ESLint 9.0 (Flat Config)	PASSED: 0 errors, 2 warnings (unused variables in page.tsx).
Backend Root API	client.get('/') (TestClient)	FastAPI 0.143.0	PASSED: HTTP 200 OK: {'message': 'MetricMind API is running'}.
Backend Summary API	client.get('/summary') (TestClient)	FastAPI 0.143.0	PASSED: HTTP 200 OK: 5,000 orders, 1.03B rev, 306.2M profit.
Backend dbt Mart API	client.get('/dbt/sales-summary')	FastAPI 0.143.0	MASKED 500: Database error safely masked without leaking info.

12.4 API Endpoint Validation and Edge Case Handling

Manual verification confirms GET / returns 200 OK, GET /summary aggregates 5,000 rows, GET /summary/month/13 returns 400 Bad Request, and non-existent products return 404.

12.5 Frontend Production Build Verification

Next.js build succeeded with exit code 0, 0 TypeScript errors, and 4 static routes prerendered.

12.6 dbt Data Integrity Assertion Results

Schema tests in metricmind_dbt/models/staging/schema.yml enforce not_null and unique constraints on order_id. Both passed during testing in 0.64s.

Chapter 13 — Results and Discussion

13.1 Verified System Capabilities

MetricMind AI demonstrates fast in-memory analytics, code-governed semantic modeling, guardrailed natural-language querying, and a conversational frontend prototype.

13.2 Qualitative Evaluation of the Conversational Semantic Paradigm

Mediating queries through a semantic layer mitigates SQL hallucinations and calculation discrepancies, providing a structured alternative to raw text-to-SQL.

13.3 Architectural Reliability and Governance Assessment

Decoupling components provides operational isolation and maintainability.

13.4 Discussion of Implementation Boundaries

The frontend currently operates in demo simulation mode, which should be connected to the live backend for production deployment.

Chapter 14 — Security and Limitations

14.1 Implemented Security Safeguards

- **1. Semantic Query Sandboxing:** The AI agent never writes raw SQL against physical tables, translating questions into SemanticQuery contracts. Credentials are kept in .env files excluded by .gitignore.
- **2. CORS Whitelisting:** FastAPI restricts Cross-Origin Resource Sharing strictly to authorized origins (http://localhost:3000 and http://127.0.0.1:3000).

- **3. Input Parameter Validation:** Path parameters are strictly typecast (e.g., month range 1-12), rejecting malformed input.
- **4. Database Exception Masking:** Database connection errors in /dbt/sales-summary mask internal exceptions, returning safe generic HTTP 500 errors.
- **5. Geographic Scope Guardrails:** The parser explicitly rejects state-level queries, preventing queries against unsupported grains.

> **[SECURITY CLARIFICATION]** Important Security Clarification: The presence of a semantic layer does NOT completely prevent SQL injection or guarantee system security. While Cube.js abstracts SQL query compilation, vulnerabilities can still emerge if REST query parameters are unvalidated, if downstream custom SQL models allow untrusted inputs, or if backend endpoints bypass the semantic layer. A semantic layer must be accompanied by strict input validation, network isolation, and authenticated endpoints.

14.2 Unimplemented and Recommended Production Controls

- **1. Authentication & Identity:** No user login, session management, OAuth2, or JWT validation is implemented.
- **2. Role-Based Access Control (RBAC):** No user roles or territorial access permissions exist; all queries access the entire dataset.
- **3. Transport Layer Security (TLS/HTTPS):** Services currently run over unencrypted HTTP in local development.
- **4. API Rate Limiting & Throttling:** Endpoints lack rate limiting, leaving them susceptible to denial-of-service abuse.
- **5. Row-Level Security (RLS):** Neither PostgreSQL nor Cube.js enforces row-level security filters based on caller identity.
- **6. Agent-to-Cube Token Enforcement:** In ai_agentllm/tools.py, HTTP POST requests to /cubejs-api/v1/load are dispatched without an Authorization header containing CUBEJS_API_SECRET.

14.3 Database Access and Exception Masking

Database access is restricted to queries over pre-aggregated marts. Database errors return a masked HTTP 500 error.

14.4 Technical Limitations and Dimensional Gaps

The current mart aggregates at Country/Region level, omitting State-level data. The frontend chat handler runs in simulated response mode. Cube's Sales.js model does not expose orderDate, creating a grain mismatch with tools.py.

Chapter 15 — Future Enhancements

- **1. Direct Frontend-to-Backend Wire-Up:** Replace simulated 1-second timeout in page.tsx:handleSend with live fetch calls to FastAPI and Cube.
- **2. Generative LLM Function Calling:** Integrate LangChain, LlamaIndex, or Google Gemini function calling for flexible natural language queries.
- **3. Multi-Grain Data Marts:** Materialize state-level and daily grain marts in dbt to support granular geographic drill-downs.
- **4. Role-Based Access Control (RBAC):** Add JWT authentication and row-level security in Cube.js for territorial access restrictions.
- **5. Automated CI/CD Pipelines:** Implement GitHub Actions workflows to automate dbt tests, unit tests, and Next.js builds on pull requests.

Chapter 16 — Conclusion

16.1 Summary of Contributions

MetricMind AI establishes a governed architectural blueprint for conversational business intelligence by pairing dbt transformations, Cube.js semantic metric modeling, a FastAPI gateway, a guardrailed AI Agent, and a Next.js 16 dynamic visualizer.

16.2 Strategic Impact on Enterprise Business Intelligence

Centralizing formulas in the semantic layer eliminates metric drift, while semantic query contracts protect database integrity from text-to-SQL hallucinations.

16.3 Final Assessment

The prototype validates the viability of conversational semantic BI. Automated test suites verify 100% pass rates across unit tests, production builds compile with zero errors, and the multi-tier architecture demonstrates clean separation of concerns.

References and Appendices

Official Technology Documentation References

- **1. FastAPI Framework:** Tiangolo, S. FastAPI Documentation: High performance, easy to learn, fast to code. Available: https://fastapi.tiangolo.com/
- **2. Cube.js Semantic Layer:** Cube Dev, Inc. Cube Documentation: The Semantic Layer for AI and BI. Available: https://cube.dev/docs
- **3. dbt (Data Build Tool):** dbt Labs. dbt Documentation: Transform data in your warehouse. Available: https://docs.getdbt.com/
- **4. Next.js Framework:** Vercel, Inc. Next.js Documentation: The React Framework for the Web. Available: https://nextjs.org/docs
- **5. Apache ECharts:** Apache Software Foundation. Apache ECharts: An Open Source JavaScript Visualizing Library. Available: https://echarts.apache.org/
- **6. PostgreSQL Relational Database:** The PostgreSQL Global Development Group. PostgreSQL 16 Documentation. Available: https://www.postgresql.org/docs/
- **7. Pydantic Data Validation:** Colvin, S. et al. Pydantic: Data validation using Python type annotations. Available: https://docs.pydantic.dev/
- **8. Pandas Data Analysis Library:** The Pandas Development Team. pandas: powerful Python data analysis toolkit. Available: https://pandas.pydata.org/docs/
- **9. Uvicorn ASGI Server:** Kluyver, T. et al. Uvicorn: The lightning-fast ASGI server. Available: https://www.uvicorn.org/
- **10. Tailwind CSS:** Tailwind Labs. Tailwind CSS Documentation: Utility-first CSS framework. Available: https://tailwindcss.com/docs

Appendix A: Complete Repository Audit and File Inventory

Relative File Path	Category / Tier	Purpose & Content Summary
backend/main.py	Backend API	FastAPI service with pandas CSV analytics and PostgreSQL dbt mart endpoints
backend/data/MetricMind_Corporate_Sales_Dataset.csv	Data Store	5,000 corporate sales records with financial and dimensional attributes
cube/cube.js	Semantic Layer	Entry point configuration for the Cube.js analytics server

cube/model/Sales.js	Semantic Layer	Defines 6 measures and 6 dimensions over public.sales_summary
metricmind_dbt/dbt_project.yml	Transformation	dbt project configuration and view materialization rules
metricmind_dbt/models/staging/stg_raw_sales.sql	Transformation	Cleans and types source sales data
metricmind_dbt/models/staging/schema.yml	Transformation	Source definitions and primary key tests for raw data
metricmind_dbt/models/marts/sales_summary.sql	Transformation	Aggregated dimensional sales summary mart
metricmind_dbt/models/marts/schema.yml	Transformation	Documentation and column metadata for the sales mart
ai_agentllm/schemas.py	AI Agent	Pydantic model for structured semantic queries
ai_agentllm/parser.py	AI Agent	Natural-language intent parser with state filter guardrails
ai_agentllm/tools.py	AI Agent	Dynamic calendar range calculator and Cube.js REST dispatcher
ai_agentllm/reasoning.py	AI Agent	Mathematical variance, percentage delta, and trend evaluator
ai_agentllm/test_parser.py	Test Suite	Unit tests verifying natural language parsing rules
ai_agentllm/test_tool.py	Test Suite	Unit tests verifying Cube payload formatting and date ranges
ai_agentllm/test_reasoning.py	Test Suite	Unit test script verifying variance and trend calculations
frontend/app/page.tsx	Frontend	Conversational analytics interface with interactive charts and inspection modals
frontend/app/components/charts/DynamicChart.tsx	Frontend	Dynamic Apache ECharts component rendering bar and line series
frontend/package.json	Frontend	Dependency manifest for Next.js, React, Tailwind, and ECharts
reports/MetricMind_AI_Project_Report.md	Documentation	Comprehensive Markdown project report
reports/MetricMind_AI_Project_Report.docx	Documentation	Professional Microsoft Word academic project report
reports/MetricMind_AI_Project_Report.pdf	Documentation	High-resolution PDF academic project report
reports/README.md	Documentation	Guide and inventory for generated report artifacts
reports/diagrams/system_architecture.mmd	Diagram	Editable Mermaid source for system architecture
reports/diagrams/data_flow.mmd	Diagram	Editable Mermaid source for data flow
reports/diagrams/analytics_pipeline.mmd	Diagram	Editable Mermaid source for analytics pipeline
reports/diagrams/ai_agent_workflow.mmd	Diagram	Editable Mermaid source for AI agent workflow
reports/diagrams/database_er.mmd	Diagram	Editable Mermaid source for database ER model

Appendix B: Sanitized Environment Configurations

Root .env.example:

```
APP_ENV=development
PORT=8000
API_BASE_URL=http://localhost:8000
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/metricmind
```

Cube .env.example:

```
CUBEJS_DB_TYPE=postgres
CUBEJS_DB_HOST=localhost
CUBEJS_DB_PORT=5432
CUBEJS_DB_NAME=metricmind
CUBEJS_DB_USER=postgres
CUBEJS_DB_PASS=your_password_here
CUBEJS_API_SECRET=your_cube_secret_here
```

Appendix C: Technical Glossary of Core BI & AI Terminology

- **Semantic Layer:** A code representation of corporate data that standardizes metrics and dimensions independent of physical storage schemas.
- **Measure:** A quantitative, numerical value that can be aggregated mathematically (e.g., sum of revenue, count of orders, average margin).
- **Dimension:** A qualitative categorical attribute used to group, slice, or filter measures (e.g., region, country, product category, fiscal quarter).
- **dbt (data build tool):** A transformation development framework combining SQL and software engineering best practices.
- **Data Mart:** A focused subset of a data warehouse designed for a specific business line or department.
- **ELT (Extract, Load, Transform):** Modern data pipeline pattern extracting and loading raw data directly before transformations occur.
- **Cube.js:** An open-source headless semantic layer used to build data applications and compile SQL.
- **Pydantic:** Python data validation library using type annotations to enforce runtime data contracts.
- **Apache ECharts:** Powerful, interactive open-source charting and visualization engine developed by Apache.
- **CORS:** Cross-Origin Resource Sharing security mechanism controlling web client resource access.